



M-Tech Production & Marketing

Technology | Training | Marketing | Projects

Week 3 - Day 6 Project Report

Internship Program — Batch 2026

Submitted By:	Ghazi Muhammad Abdullah – AI26025
Department:	AI / ML
Submitted To:	Malik Mohazin Zahid
Date:	18 July 2026

1. Introduction

The Handwritten Digit Recognizer is a lightweight desktop application developed using Python, NumPy, Pillow, and Tkinter to recognize handwritten digits ranging from **0 to 9**. The project demonstrates the practical implementation of a Convolutional Neural Network (CNN) without relying on popular deep learning frameworks such as TensorFlow or PyTorch. Instead, the complete neural network was implemented from scratch using only NumPy, allowing a deeper understanding of the internal working of convolutional neural networks.

The application provides an interactive Tkinter-based graphical interface where users can draw a digit using the mouse on a black canvas. Once the drawing is completed, the CNN processes the image and predicts the handwritten digit along with its confidence percentage. To ensure offline functionality, a bundled subset of the MNIST handwritten digit dataset is included within the project. During the first execution, the model is automatically trained and the learned weights are stored locally. On future executions, the saved model is loaded directly, eliminating the need for retraining and improving application startup time.

2. Objectives

- Develop a desktop application capable of recognizing handwritten digits using a Convolutional Neural Network (CNN).
- Implement the complete CNN architecture from scratch using only NumPy.
- Build an interactive graphical interface using Tkinter for digit drawing and prediction.
- Train the model using a bundled subset of the MNIST handwritten digit dataset.
- Automatically save trained model weights for future use.
- Perform handwritten digit prediction completely offline without requiring an internet connection.
- Demonstrate the complete workflow of data preprocessing, CNN training, model evaluation, and prediction.

3. Task Description

The project involved designing and implementing a complete desktop-based handwritten digit recognition system using Python. A balanced subset of the MNIST handwritten digit dataset was bundled with the project to eliminate the need for downloading data during execution.

The CNN architecture was developed entirely with NumPy and consisted of convolution, ReLU activation, max pooling, flattening, fully connected layers, and softmax output for classification. Cross-entropy loss and mini-batch gradient descent were implemented for model optimization during training.

A Tkinter graphical interface was created to allow users to draw handwritten digits on a canvas. The application preprocesses the drawing, converts it into the required input format, and passes it through the trained CNN model to generate predictions. The predicted digit and confidence

percentage are displayed immediately. The trained model is automatically saved after the first execution and loaded in subsequent runs to improve efficiency.

4. Work Done

The project was completed by following the complete machine learning development process. Initially, the project structure was organized into separate folders for source code, dataset, trained models, documentation, and application files to maintain clean project architecture.

A complete Convolutional Neural Network was implemented from scratch using NumPy, including Conv2D layers, ReLU activation, MaxPooling, Flatten, Dense layers, Softmax activation, and Cross-Entropy Loss. Image preprocessing routines were developed to resize, normalize, and prepare user-drawn digits before prediction.

A user-friendly Tkinter interface was then designed with a drawing canvas, Predict button, Clear button, and result display labels. Automatic model training was implemented for the first application launch, while trained weights were cached locally inside the models directory for faster future execution. Additional project files such as **check_setup.py**, **requirements.txt**, and **README.md** were also prepared to simplify installation and execution across different systems.

5. Challenges Faced

Several technical challenges were encountered during the development of this project. One of the primary challenges was implementing an efficient Convolutional Neural Network entirely with NumPy without using any external deep learning frameworks. Building convolution operations, pooling layers, forward propagation, backpropagation, and parameter updates required careful mathematical implementation and extensive debugging.

Another challenge involved preprocessing the user-drawn images to closely match the format of the MNIST training dataset so that prediction accuracy remained consistent. Managing automatic model saving and loading while ensuring compatibility between different executions also required additional testing. The graphical interface was refined through multiple iterations to provide smooth drawing functionality and responsive prediction results.

6. Key Learnings

This project provided practical experience in implementing deep learning algorithms from the ground up without relying on high-level machine learning libraries. I gained a stronger understanding of convolution operations, feature extraction, activation functions, pooling techniques, fully connected neural networks, softmax classification, cross-entropy loss, and gradient descent optimization.

The project also enhanced my Python programming skills by integrating machine learning with desktop application development using Tkinter. I learned how to preprocess image data, organize large Python projects into modular components, cache trained models, and create an interactive application that operates efficiently in a completely offline environment. Overall, the project significantly improved my understanding of both deep learning concepts and software engineering practices.

7. Conclusion

The Handwritten Digit Recognizer successfully demonstrates the practical implementation of a Convolutional Neural Network for image classification in a lightweight desktop application. By combining a custom-built CNN, efficient image preprocessing, and a simple Tkinter graphical interface, the system is capable of accurately recognizing handwritten digits while remaining easy to install and operate.

The project integrates data preparation, neural network implementation, model training, prediction, and graphical interface development into a complete artificial intelligence solution. It serves as an excellent educational demonstration of deep learning fundamentals and provides a strong foundation for future enhancements such as support for larger datasets, improved CNN architectures, real-time handwriting recognition, and expanded image classification capabilities.

Intern Signature:



Supervisor:

Name: Ghazi Muhammad Abdullah

Name:

Date: 18 July 2026

Designation: